



TraceProv: All You Need is Log - Efficient Bolt-on Provenance Capture for Complex Relational Queries

Vinayak Jha Boris Glavic

University of Illinois at Chicago



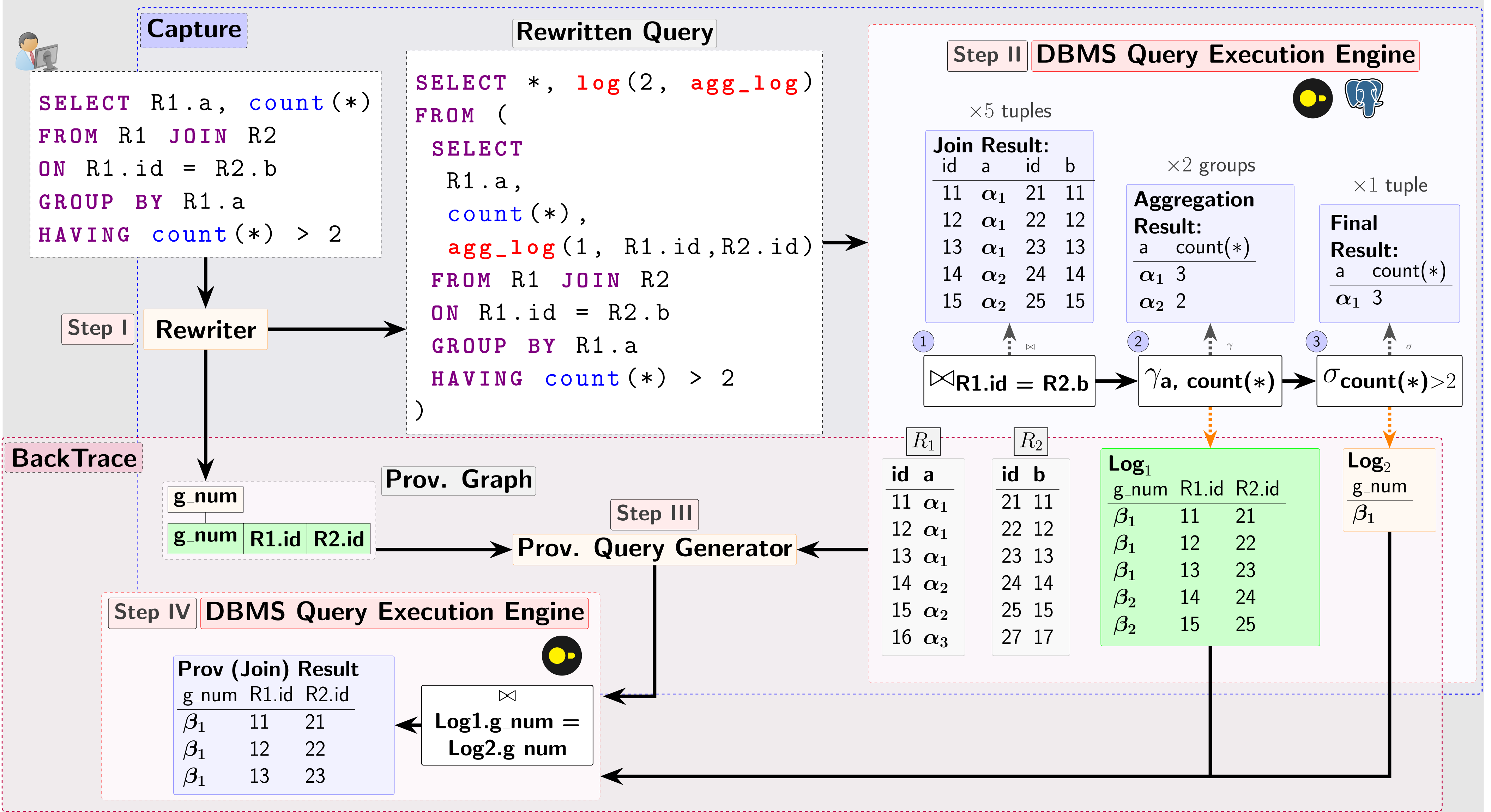
Background and Motivation

- **Data Lineage** maps query result tuples to the contributing tuples in the database.
- **GProM [1]** (*Database agnostic*; 1 Phase)
 - Middleware; Rewrites user queries to capture lineage
 - Propagates provenance bottom-up $\Rightarrow$ No factorization possible
 - **Low** entanglement, **low** performance
- **ProvSQL [2]** (*PostgreSQL*; 2 Phase)
 - PostgreSQL extension; Represents, and stores, lineage via **circuits** in **memory-mapped files**
 - Allows factorization
 - **I**: Stores provenance circuit (DAG representation of provenance expressions)
 - **II**: Reconstructs provenance by evaluating stored circuits bottom up
 - **Medium** entanglement, **low** performance
- **SQLProv [3]** (*PostgreSQL*; 2 Phase)
 - PostgreSQL-based system; Logs, and propagates, provenance tokens throughout query execution
 - Allows factorization
 - **I**: Logs value-based decisions during query evaluation
 - **II**: Reconstructs provenance by replaying query execution using log tables from input $\rightarrow$ query output
 - **Low** entanglement, **low** performance
- **SmokedDuck [4]** (*DuckDB*; 2 Phase)
 - Modified DuckDB system;
 - **I**: Logs provenance of individual operators
 - **II**: Reconstructs provenance by joining log tables (using zonemaps), from query output $\rightarrow$ input
 - **High** entanglement, **high** performance
- **Database-agnostic approach with high performance and reasonable entanglement?**

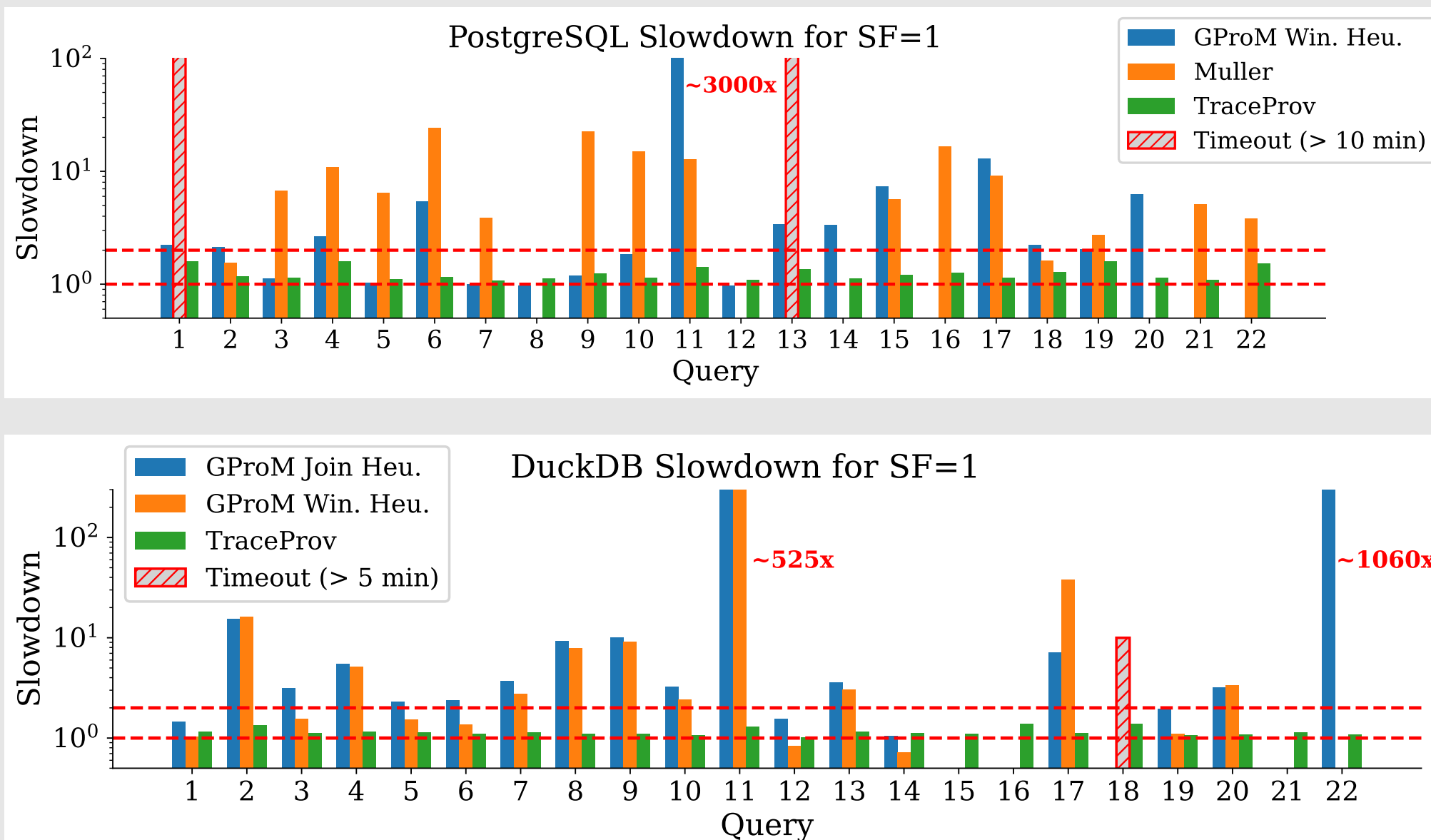
TraceProv

- **Annotation propagation vs. logging mapping tables**
 - **Annotation propagation**: $-$ blows up intermediate result size following aggregation $+$ single phase $+$ no storage overhead
 - **Provenance mapping tables**: $+$ steady intermediate result size $-$ storage overhead $-$ requires reconstruction phase
 - **TraceProv**: **best of both worlds** - Opportunistically combines propagation with logging
- **Top-down backtrace during provenance reconstruction**
 - TraceProv joins provenance mapping tables top-down (query output $\rightarrow$ input)
 - Avoids useless work performed in bottom-up approaches to construct provenance for intermediate results that eventually get filtered out
- **Append-only logging framework**
 - Top-down construction allows efficient append-only logging
 - Supports logging to mmap'ed files, memory pages, huge pages
- **Parallelism-friendly**
 - Avoids generating globally stable identifiers (uses rowids / primary keys)
 - Does not impede parallel execution, and scales well
- **Maintains original query structure**
- **Lies to database where it is safe**
 - Selectively lies to the database about stability of logging functions when sufficient information is guaranteed to be logged
- **Low entanglement**
 - Implemented entirely via extensibility mechanisms supported by most databases
 - Supports DuckDB and PostgreSQL

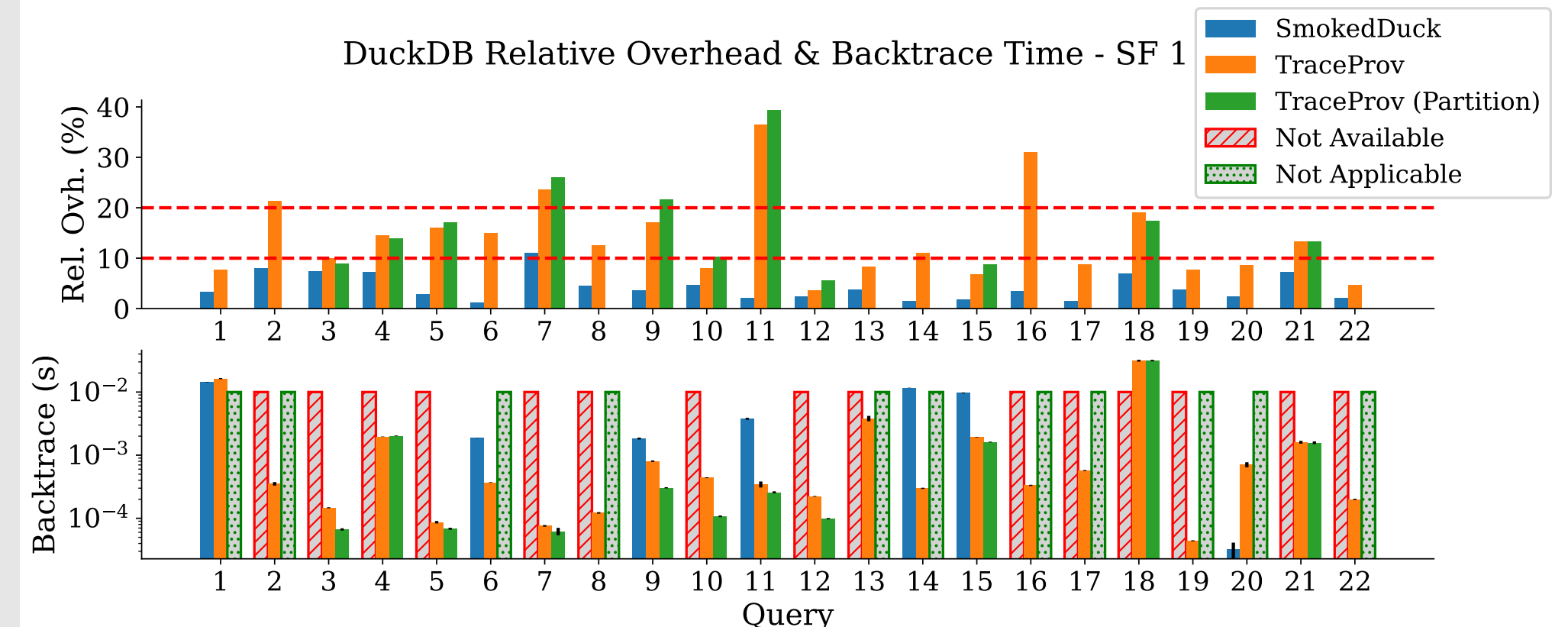
TraceProv Example



Experimental Results (TPC-H)



Experimental Results (TPC-H) Contd.



References

- [1] Bahareh Arab et al. "GProM - A Swiss Army Knife for Your Provenance Needs". In: *IEEE Data Eng. Bull.* 41.1 (2018), pp. 51–62.
- [2] Pierre Senellart et al. "ProvSQL: provenance and probability management in postgresQL". In: *PVLDB* 11.12 (2018), pp. 2034–2037.
- [3] Tobias Müller, Benjamin Dietrich, and Torsten Grust. "You say 'what', i hear 'where' and 'why': (mis-) interpreting SQL to derive fine-grained provenance". In: *PVLDB* 11.11 (2018), pp. 1536–1549.
- [4] Haneen Mohammed et al. "SmokedDuck Demonstration: SQLStepper". In: *SIGMOD*. Ed. by Sudipto Das et al. 2023, pp. 183–186.